CS5200 Final Report

Jessica Pham, Shubham Madhusudan Hadawle
Group: PhamJHadawleS

March 12, 2026

1 Project Description

We would like to propose a cocktail database with which a user can look up recipes of a specific name, flavor(s), liquor type, number of ingredients, or difficulty through a website we will call MixMaster. The user should also be able to favorite cocktails and post reviews. Each cocktail entry will display a list of ingredients with quantities, tools needed, preparation instructions, flavor tags, recommended glass type, and a photo. The application distinguishes between "simple" (where ingredients are merely combined) and "complex" cocktails (which require a specific sequence of preparation steps).

We chose this topic because we wanted to have a little fun! The primary goal is to provide an intuitive, searchable interface for beginners looking for easy recipes as well as for bartenders exploring new combinations.

1.1 Tech Stack

Front end: Javascript
Back end: Python
Database: MySQL
Database: mySQL
Version Control: Git / Github
Frameworks: React.js, FastAPI
Machine Restrictions: None.
The application should be platform-independent.

1.2 Application Functionalities

- Users should be able to see ingredients and flavors for each cocktail.
- Users should be able to login/register and favorite cocktails.
- Users should be able to post reviews for cocktails.
- Users should be able to query cocktails based on flavors and if its difficulty is simple or complex.
  - "simple" cocktails just involve mixing of all ingredients
  - "complex" cocktails require a specific sequence of steps
- Users should be able to query based on fewest/most number of ingredients.
- Users should be able to query based on liquor(s) (ingredient type)
- Each cocktail should have a picture and instructions along with a list of tools that cocktail needs.
- Each cocktail should also show which glass type is usually used for that cocktail if applicable.

2 Database Design

2.1 Entities and Attributes

The database is organized around the following core entities:

Cocktail: The central entity. Each cocktail has a unique identifier, a name, a text description, a difficulty classification (simple or complex), an image URL, and timestamp fields for record creation and modification. Each cocktail is associated with exactly one glass type.

GlassType: Represents the style of glassware recommended for serving a cocktail (e.g., Coupe, Highball, Rocks). Each glass type has a unique identifier, a name, a description, and an optional image URL.

Recipe: Each Cocktail has exactly one recipe which contains instructions and describes the difficulty of the recipe. The recipe lists the tools and ingredients needed to make the cocktail.

Ingredient: Represents an individual ingredient that may appear in many recipes. Each ingredient has a unique identifier, a name, an optional description, and a foreign key linking it to its ingredient type.

IngredientType: Categorizes ingredients into groups such as Spirit, Mixer, Garnish, Syrup, Juice, or Bitters. This entity enables users to filter cocktails by liquor (i.e., ingredients of type "Spirit"). Each type has a unique identifier, a name, and a description.

Flavor: Represents a flavor profile tag (e.g., Sweet, Sour, Bitter, Smoky, Herbal). Each flavor has a unique identifier, a name, and a description.

AppUser: Represents a registered user of the application. Each user has a unique identifier, a unique username, a unique email address, a hashed password, and a registration timestamp.

Review: Users can create reviews for cocktails they have tried, giving it a 1-5 star rating and a text description. These reviews should be shown on the bottom of the page of the specific cocktail page.

Tool: For complex cocktails/recipes, there may be need for multiple tools or equipment such as muddlers, shakers, strainers, etc.

2.2 Proposed Schema

- glass_type(**glass_type_id INTEGER**, name, VARCHAR(), description VARCHAR(), image_url VARCHAR())
  - **nullable**: description, image_url
- ingredient_type(**ingredient_type_id INTEGER**, name VARCHAR(), description())
  - **AK:** name
  - **nullable:** description
- ingredient(**ingredient_id INTEGER**, name VARCHAR(), description VARCHAR(), ingredient_type_id INTEGER)
  - **AK:** name
  - **FK:** ingredient_type_id → ingredient_type(ingredient_type_id)
  - **nullable:** description
- cocktail(**cocktail_id INTEGER**, name VARCHAR(), description VARCHAR(), image_url VARCHAR(), glass_type_id INTEGER, created_at DATETIME, updated_at DATETIME, recipe_id INTEGER)
  - **AK:** name
  - **FK:** glass_type_id → glass_type(glass_type_id)
  - **FK:** recipe_id → recipe(recipe_id)
  - **nullable**: description, image_url

- recipe(**recipe_id INTEGER**, instructions VARCHAR(), difficulty ENUM('simple', complex')
  - **difficulty**: "simple" xor "complex"
- tool(**tool_id INTEGER**, name VARCHAR(), description VARCHAR(), image_url VARCHAR())
- recipe_tool(**recipe_tool_id INTEGER**, recipe_id INTEGER, tool_id INTEGER)
  - AK: recipe_id, tool_id
  - **FK:** recipe_id → recipe(recipe_id)
  - **FK:** tool_id → tool(tool_id)
- recipe_ingredient(**recipe_ingredient_id INTEGER**, recipe_id INTEGER, ingredient_id INTEGER, quantity DOUBLE, unit VARCHAR())
  - **AK**: recipe_id, ingredient_id
  - **FK:** recipe_id → recipe(recipe_id)
  - **FK:** ingredient_id → ingredient(ingredient_id)
  - **nullable:** unit (some ingredients may not have a formal unit)
- flavor(**flavor_id INTEGER**, name VARCHAR(), description VARCHAR())
  - **AK:** name
  - **nullable:** description
- cocktail_flavor(**cocktail_flavor_id INTEGER**, cocktail_id INTEGER, flavor_id INTEGER)
  - **AK**: cocktail_id, flavor_id
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **FK:** flavor_id → flavor(flavor_id)
- app_user(**user_id INTEGER**, username VARCHAR(), email VARCHAR(), password_hash VARCHAR(), created_at DATETIME)
  - **AK:** username
  - **AK:** email
  - **password_hash:** should store hashed value, never plaintext
  - **created_at:** default CURRENT_TIMESTAMP
- user_favorite(**favorite_id INTEGER**, user_id INTEGER, cocktail_id INTEGER, created_at DATETIME)
  - **AK**: user_id, cocktail_id
  - **FK:** user_id → app_user(user_id)
  - **FK:** cocktail_id → cocktail(cocktail_id)
  - **created_at:** default CURRENT_TIMESTAMP
- review(**review_id INTEGER**, user_id INTEGER, cocktail_id INTEGER, rating INTEGER, review_text VARCHAR(), created_at DATETIME)
  - **AK**: user_id, cocktail_id
  - **FK**: user_id → app_user(user_id)
  - **FK**: cocktail_id → cocktail(cocktail_id)
  - **rating**: 1–5
  - **nullable**: review_text

2.3 Relationships

GlassType → Cocktail: Each Cocktail is served in exactly one GlassType, representing the recommended serving vessel (e.g. Coupe, Highball, Rocks glass). A single GlassType may be associated with zero or many Cocktails. Mandatory on the Cocktail side and optional on the GlassType side.

Cocktail → Recipe: Each Cocktail has exactly one Recipe. Each Recipe belongs to exactly one Cocktail. Mandatory on both sides — a cocktail must have a recipe, and a recipe must belong to a cocktail.

Recipe → Ingredient: A Recipe requires one or more Ingredients. An Ingredient can appear in many different Recipes. RecipeIngredient associates these entities and stores the quantity and unit of measure for each ingredient in a given recipe. Mandatory on the Recipe side and optional on the Ingredient side.

Recipe → Tool: A Recipe may need zero or more Tools (e.g. muddler, shaker, strainer). A Tool can be needed by zero or many Recipes. RecipeTool associates these entities. Optional on both sides — a simple recipe may require no special tools, and a tool may exist in the database before being assigned to any recipe.

Ingredient → IngredientType: Each Ingredient is classified as exactly one IngredientType (e.g. Spirit, Mixer, Garnish). An IngredientType can classify one or more Ingredients. Mandatory on the Ingredient side and optional on the IngredientType side.

Cocktail → Flavor: A Cocktail can be tagged with one or more Flavors. A Flavor can be assigned to zero or many Cocktails. CocktailFlavor associates these entities. Mandatory on the Cocktail side and optional on the Flavor side.

Cocktail → Review: A Cocktail can have zero or many Reviews. Each Review belongs to exactly one Cocktail. Mandatory on the Review side and optional on the Cocktail side — a cocktail may exist without any reviews yet.

AppUser →Review: An AppUser can create zero or many Reviews. Each Review is created by exactly one AppUser. Mandatory on the Review side and optional on the AppUser side — a user may exist without having written any reviews.

AppUser → Cocktail (Favorites): An AppUser can favorite zero or many Cocktails. A Cocktail can be favorited by zero or many AppUsers. UserFavorite associates these entities and records the timestamp of when the cocktail was favorited. Optional on both sides.

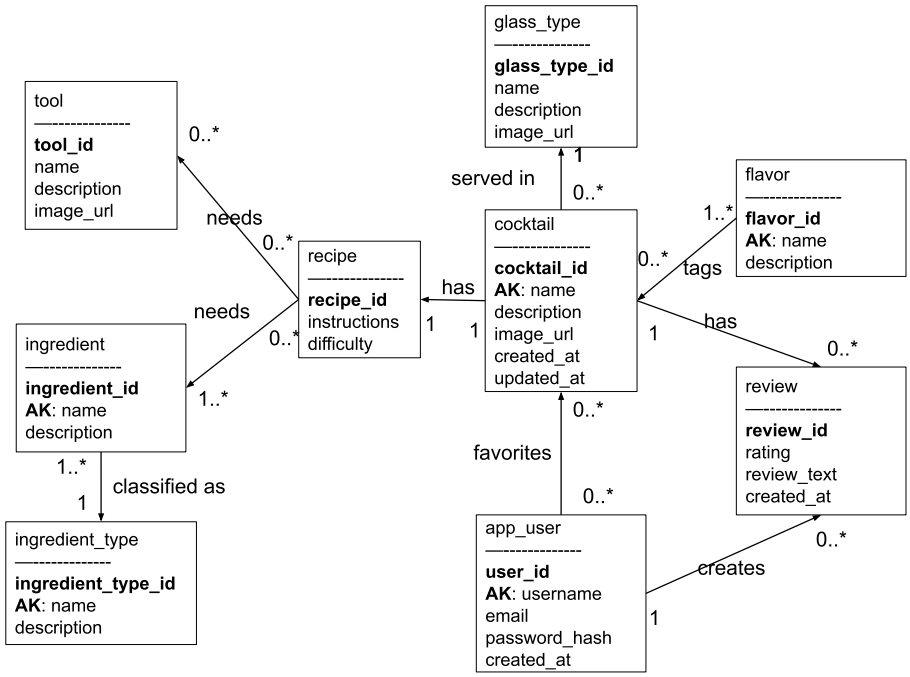

Figure 1: UML Diagram

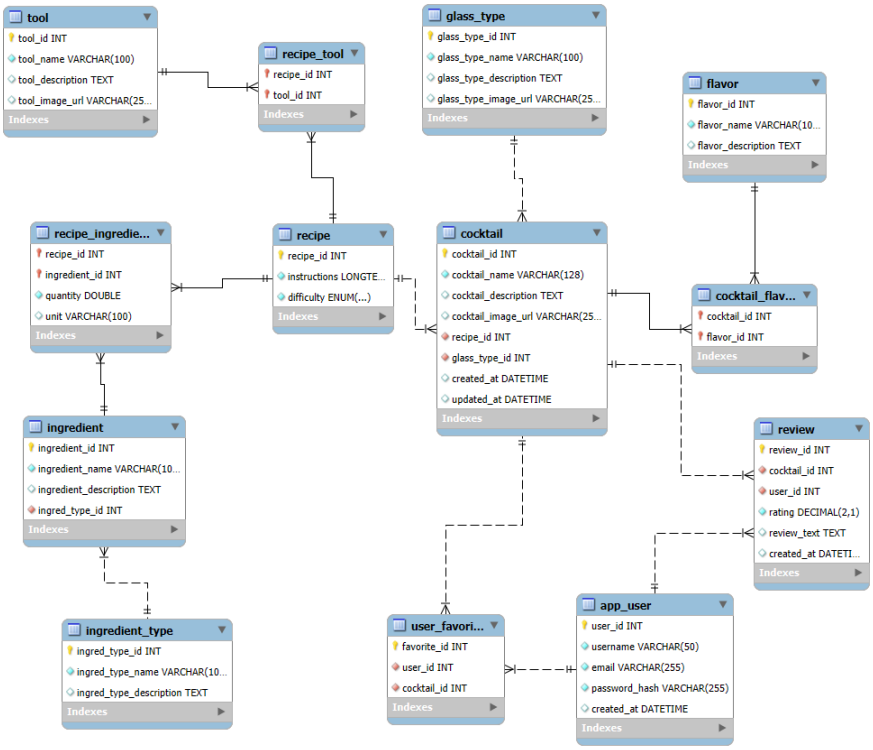

Figure 2: Logical Design

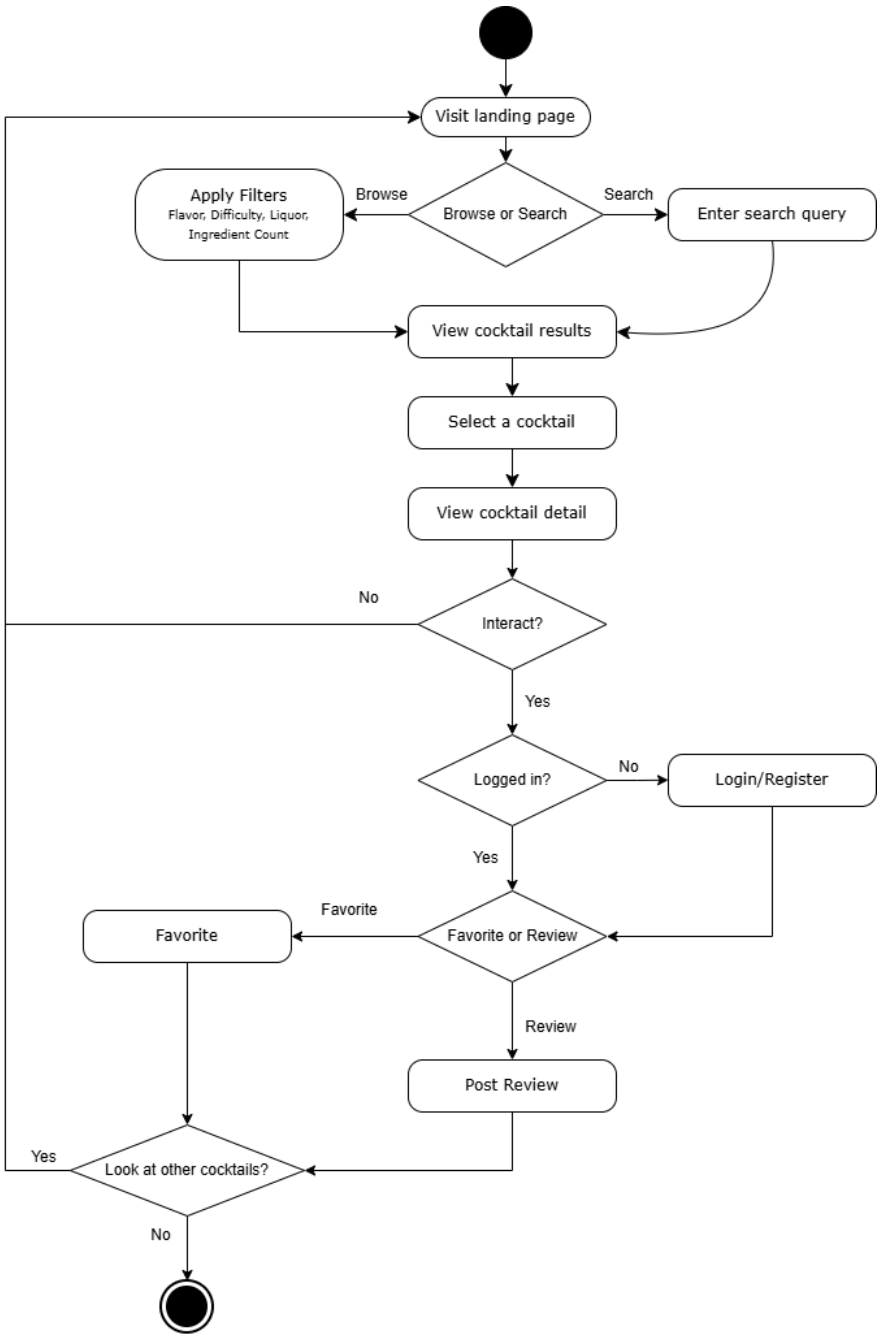

Figure 3: Activity Diagram